

Software Engineering (CS24309) — Full Course Master Book

Complete End-to-End B.Tech CSE 5th Semester Study Book & PYQ Bank

EXECUTIVE MASTER STUDY GUIDE & PROJECT METRICS BANK

Software Engineering (CS24309)

Birla Institute of Technology, Mesra | B.Tech CSE 5th Semester (NEP 2024–25 Scheme)

Complete Course Structure & Software Engineering Matrix

Module	Core Scope	Key Metrics, Methodologies & Formulations
Module I	Process Models & Agile	Waterfall, V-Model, Prototyping, Boehm's Spiral (4 Quadrants), Agile Manifesto, Scrum Roles & Sprint Burndown
Module II	Requirements & Analysis	IEEE 830 SRS Standard, Use Case Modeling, Data Flow Diagrams (DFD Level 0/1/2), Data Dictionary & ER Diagrams
Module III	Design & Architecture	Cohesion (Functional to Coincidental), Coupling (Data to Content), SOLID Principles, GoF Design Patterns
Module IV	Metrics & Estimation	Albrecht Function Points (FP), Basic & Intermediate COCOMO Models, PERT Expected Time & CPM Critical Path
Module V	Testing & Maintenance	Black-Box (ECP, BVA), White-Box (Basis Path & McCabe's Cyclomatic Complexity), Mutation Testing, Maintenance

Exam Preparation & High-Yield Strategy

This publication-grade master book consolidates all 5 modules with step-by-step COCOMO & PERT cost estimation calculations, Cyclomatic Complexity graph proofs, and model answers to BIT Mesra end-semester examination questions.

☰ Module I: Introduction & Process Models — Complete 14-Topic Syllabus Tracker

Topic 1: What is Software Engineering? (Systematic Discipline)

Topic 2: Why Software Engineering? (Software Crisis & Complexity)

Topic 3: The Evolving Role of Software (Product, Service & System)

Topic 4: Software Process Framework (Core Generic Activities)

Topic 5: Classical Waterfall Model (Linear Sequential Lifecycle)

Topic 6: Prototyping Model (Requirement Discovery & User Feedback)

Topic 7: Boehm's Spiral Model (Risk-Driven Iterative Cycles)

Topic 8: Rapid Application Development (RAD Component Reuse)

Topic 9: Incremental Process Model (Progressive Feature Delivery)

Topic 10: Agile Models (Agile Manifesto & Sprint Feedback Loops)

Topic 11: Software Management Activities (The 4 P's Framework)

Topic 12: Project Planning (Scope, Cost & Resource Allocation)

Topic 13: Project Scheduling (WBS, Gantt Charts & PERT/CPM)

Topic 14: Risk Management (I-A-P-M-M Risk Assessment Cycle)

Topic 1 – 3: What is Software Engineering & The Software Crisis?

Software Engineering (IEEE 610.12) is the systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering principles to software.

Ad-Hoc Programming	Software Engineering Discipline
Individual developer writing code without formal specifications.	Multi-person, multi-team systematic engineering development lifecycle.
No formal documentation; maintainability ignored.	Comprehensive SRS, architectural designs, test plans, and version control.
Massive budget overruns, missed deadlines, high defect rates.	Predictable cost estimation (COCOMO), scheduling (PERT/Gantt), and QA/QC.

Topic 4 – 10: Software Development Process Models

Process Model	Core Lifecycle Mechanism & Characteristics	Ideal Project Suitability
1. Waterfall Model	Strict linear sequential phases (Requirements → Design → Code → Test → Deploy). Each phase produces sign-off deliverables before the next begins.	Well-understood, stable requirements with mature technology stacks (e.g., Space flight software, banking backends).
2. Prototyping	Builds an early disposable mock-up interface to elicit feedback, refine ambiguous user requirements, and discard/re-engineer.	High user interaction systems where clients have unclear or fuzzy initial requirements.
3. Spiral Model (Boehm)	Risk-driven iterative model structured in 4 quadrants: (1) Objective Determination, (2) Risk Analysis , (3) Engineering, (4) Customer Evaluation.	Large-scale, expensive, mission-critical, high-risk enterprise projects.
4. RAD Model	Rapid Application Development (60–90 days) utilizing component-based construction, GUI builders, and automated code generators.	Well-modularized information systems with short delivery deadlines and high component reuse.
5. Incremental Model	System is decomposed into modular release increments. Increment 1 delivers core functionality; subsequent releases add peripheral features.	Projects requiring early partial operational software release to capture market share.
6. Agile (Scrum / XP)	Iterative 2–4 week Sprints emphasizing working software over documentation, customer collaboration, and adaptive response to changing requirements.	Dynamic web/mobile products with volatile, continuously evolving market requirements.

🧠 Memory Hook: Process Model Selection Strategy

- Stable requirements → **Waterfall**
- Unclear user requirements → **Prototyping**
- High financial/technical risk → **Spiral**
- Rapid component delivery → **RAD / Incremental**
- Changing, dynamic requirements → **Agile**

Topic 11 – 14: Project Planning, Scheduling & Risk Management

The 5–Stage Risk Management Cycle (I–A–P–M–M)

1. **Risk Identification:** Identify project, product, and business risks (e.g., staff turnover, scope creep).
2. **Risk Analysis (Estimation):** Quantify Risk Probability (P) and Risk Impact / Severity (I). **Risk Exposure** (RE) = $P \times I$.
3. **Risk Prioritization:** Rank risks in a Risk Matrix to focus engineering resources on top hazards.
4. **Risk Mitigation (RMMM Plan):** Develop Risk Mitigation, Monitoring, and Management strategies to minimize impact.
5. **Risk Monitoring:** Continuously track risk indicators throughout the project lifecycle.



M1 Active Recall & Exam Questions

Q1. Compare Waterfall, Prototyping, Spiral, and Agile models across 5 engineering parameters. (10 Marks)

1. **Requirement Flexibility:** Waterfall is rigid; Prototyping clarifies fuzzy needs; Spiral adapts per cycle; Agile welcomes changes at any stage.
2. **Risk Management:** Spiral explicitly incorporates formal Risk Analysis in every cycle; Waterfall and Prototyping have zero built-in risk analysis; Agile manages risk via short sprint iterations.
3. **Customer Involvement:** Waterfall involves customer only at start and end; Agile and Prototyping require continuous customer collaboration.
4. **Delivery Schedule:** Waterfall delivers product only at the very end; Incremental/Agile delivers functional software increments every few weeks.
5. **Cost & Documentation:** Waterfall is documentation-heavy; Spiral is expensive due to risk experts; Agile prioritizes working software over extensive documentation.

Module II: Software Requirements Engineering — Complete 12-Topic Syllabus Tracker

Topic 15: Functional Requirements (Core Behavioral Capabilities)

Topic 17: User Requirements (High-Level Natural Language Statements)

Topic 19: The Software Requirements Specification (SRS) Document

Topic 21: 9 Characteristics of a Good SRS (C-U-C-C-V-F-T-P-M)

Topic 23: Requirements Elicitation Techniques (Interviews, JAD, Prototyping)

Topic 25: Requirements Validation & Formal Inspection Reviews

Topic 16: Non-Functional Requirements (Quality Constraints & Performance)

Topic 18: System Requirements (Detailed Technical Specifications)

Topic 20: IEEE 830 Standard Format for SRS Organization

Topic 22: Feasibility Study (The TELOS Analytical Framework)

Topic 24: Requirements Analysis & Conflict Negotiation

Topic 26: Requirements Management & Traceability Matrices (RTM)

Topic 15 – 18: Functional vs. Non-Functional & User vs. System Requirements

Requirement Class	Formal Definition & Scope	Concrete Illustrative Example
1. Functional	Specifies WHAT services, computations, and transformations the software must perform in response to specific inputs.	"The banking system shall transfer funds between accounts and send SMS confirmation within 5 seconds."
2. Non-Functional	Specifies HOW WELL the system operates, defining constraints on quality attributes: Performance, Security, Availability, Usability.	"The system shall maintain 99.99% annual availability and encrypt database records using AES-256."
3. User Requirements	High-level natural language statements + informal diagrams intended for non-technical clients and executive sponsors.	"Patients should be able to book doctor consultations online."
4. System Requirements	Structured, unambiguous technical contracts detailing software services, inputs, outputs, exceptions, and DB schemas for developers.	"The API endpoint `POST /api/v1/appointments` shall validate doctor availability locks via Redis mutexes."

Topic 19 – 21: The SRS Document & IEEE 830 Standard

The 9 Characteristics of a High-Quality SRS (C-U-C-C-V-F-T-P-M)

1. **Correct:** Every requirement accurately reflects actual customer needs.
2. **Unambiguous:** Every requirement has exactly ONE possible interpretation by developers and testers.
3. **Complete:** All critical functional capabilities, responses to errors, and constraints are specified.
4. **Consistent:** Zero contradictions between individual requirements or interface definitions.
5. **Verifiable (Testable):** A finite, cost-effective test case can prove whether the software satisfies the requirement.
6. **Feasible:** The requirement is achievable within project budget, timeline, and technology constraints.
7. **Traceable:** Each requirement has a unique identifier linked backward to user needs and forward to code/tests.
8. **Prioritized:** Requirements are ranked by business importance (Essential, Desirable, Optional).
9. **Modifiable:** Organized with a clear table of contents and cross-references for easy updates.

Topic 22 – 26: Requirements Engineering Process (TELOS to Traceability)

The TELOS Feasibility Analysis Framework: – **T — Technical:** Do we have the hardware, software tools, and developer expertise? – **E — Economic:** Is the Return on Investment (ROI) and Cost-Benefit Analysis positive? – **L — Legal:** Does the product comply with GDPR, HIPAA, copyright, and regional labor laws? – **O — Operational:** Will end-users and the organizational structure accept the system? – **S — Schedule:** Can the product be delivered before the critical market deadline?

Requirements Traceability Matrix (RTM) Sample Architecture

Req ID	Requirement Description	Source	Design Module	Code File	Test Case ID
`REQ-01`	User OTP Authentication	Client Specs §2.1	`AuthModule`	`auth_service.py`	`TC-AUTH-001`
`REQ-02`	Payment Gateway Checkout	Client Specs §3.4	`PaymentEngine`	`payment_gateway.py`	`TC-PAY-014`

M2 Active Recall & Exam Questions

Q1. Explain the IEEE 830 standard structure for a Software Requirements Specification (SRS). (8 Marks)

The IEEE 830 standard organizes an SRS into 3 primary sections:

1. **Section 1: Introduction:** Purpose, Scope, Definitions, Acronyms, References, and Document Overview.
2. **Section 2: Overall Description:** Product Perspective, Product Functions, User Classes & Characteristics, Operating Environment, Design Constraints, Assumptions & Dependencies.
3. **Section 3: Specific Requirements:** Detailed Functional Requirements, External Interface Requirements (User, Hardware, Software, Comm), Performance Requirements, Security & Safety Attributes, and Verification Criteria.

☰ Module III: Software Design Engineering & UML — Complete 13-Topic Tracker

Topic 27: Design Engineering (Architecture vs. Detailed Component Design)

Topic 29: Software Design Quality Attributes (Modularity & Maintainability)

Topic 31: Design Models (Data, Architectural, Interface & Component)

Topic 33: Cohesion (7 Levels: Coincidental to Functional Cohesion)

Topic 35: UML Use Case Diagrams (Actors, Systems & Include/Extend)

Topic 37: UML Activity Diagrams (Workflows, Swimlanes & Fork/Join)

Topic 39: UML Collaboration / Communication Diagrams

Topic 28: The Software Design Process (Iterative Model Transformations)

Topic 30: Fundamental Design Concepts (Abstraction & Information Hiding)

Topic 32: Object-Oriented Design Principles (SOLID Design Principles)

Topic 34: Coupling (5 Levels: Content to Data Coupling)

Topic 36: UML Class Diagrams (Attributes, Methods, Visibility & Relations)

Topic 38: UML Sequence Diagrams (Lifelines, Messages & Time Sequence)

Topic 27 – 30: Fundamental Software Design Concepts

Design Concept	Engineering Principle & Definition	Key Benefit
1. Abstraction	Suppressing low-level implementation details to focus on high-level essential characteristics.	Reduces cognitive complexity for system architects.
2. Modularity	Decomposing software into discrete, independently testable, naming-addressable modules.	Enables parallel team development and isolated debugging.
3. Information Hiding	Modules are designed so that internal algorithms and data structures are inaccessible from outside (Parnas, 1972).	Prevents cascading bug propagation when internal code changes.
4. Functional Independence	Achieved by designing modules with High Cohesion and Low Coupling .	Maximizes code reusability and minimizes side-effect bugs.

Topic 33 & 34: Cohesion vs. Coupling (The Cornerstone of Modular Design)

Cohesion Level (Internal Focus)	Operating Semantics	Quality Rating
1. Functional Cohesion	Every instruction in the module contributes to executing exactly ONE single mathematical/business function (e.g., <code>calculate_tax()</code>).	★★★★★ BEST
2. Sequential Cohesion	Output data from one operation serves as direct input to the next operation in the module.	★★★★ Very Good
3. Communicational Cohesion	Functions operate on the exact same input data structure (e.g., <code>print_student_record()</code> and <code>save_student_record()</code>).	★★★ Good
4. Procedural / Temporal	Functions executed in a sequence or executed at the same time (e.g., <code>startup_initialization()</code>).	★★ Moderate
5. Coincidental Cohesion	Arbitrary unrelated tasks grouped together in a <code>utils.py</code> file with no logical relationship.	★ WORST

Coupling Level (Inter-Dependency)	Operating Semantics	Quality Rating
1. Data Coupling	Modules communicate strictly by passing necessary simple primitive data parameters (x, y).	★★★★★ BEST
2. Stamp Coupling	Modules pass entire complex data structures when only a single field is actually needed.	★★★★ Good
3. Control Coupling	One module passes a control flag (<code>bool is_admin</code>) that dictates the internal execution path of another.	★★ Moderate
4. Common Coupling	Multiple modules read and write to shared global variables.	★ Poor
5. Content Coupling	One module directly accesses, modifies, or branches into the internal memory/code of another module.	✖ WORST

🧠 Golden Rule of Software Design

HIGH COHESION (Functional) + LOW COUPLING (Data) = Publication-Grade Software Architecture!

Topic 35 – 39: Unified Modeling Language (UML) Diagrams

UML Diagram	Structural / Behavioral	Key Modeling Elements & Purpose
1. Use Case Diagram	Behavioral	Actors (stick figures), Use Cases (ovals), System boundary, `<>` (mandatory) and `<>` (optional).
2. Class Diagram	Structural	Classes with 3 compartments: Name, Attributes (`- private`, `+ public`, `# protected`), Operations. Inheritance ($\triangle$), Association, Composition ($\blacklozenge$).
3. Activity Diagram	Behavioral	Initial/Final nodes, Action states, Decision diamonds, Swimlanes, Fork/Join synchronization bars.
4. Sequence Diagram	Behavioral (Dynamic)	Lifelines (vertical dashed lines), Activation boxes, Synchronous solid-arrow calls, Asynchronous open-arrow calls, Return dashed arrows.
5. Collaboration Diagram	Behavioral (Dynamic)	Emphasizes structural organization of objects with numbered message arrows (1.0, 1.1) rather than a vertical time axis.

M3 Active Recall & Exam Questions

Q1. Differentiate between `<>` and `<>` relationships in UML Use Case Diagrams with examples. (8 Marks)

- `<>` Relationship: Represents mandatory common functionality unconditionally shared across multiple use cases. The base use case *cannot execute* without invoking the included use case.

Example: `PlaceOrder` $\rightarrow$ {>} `AuthenticateUser`.

- `<>` Relationship: Represents optional, conditional behavior that executes only when a specific extension point condition is met at runtime.

Example: `CalculateTotal` $\leftarrow$ {>} `ApplyDiscountCoupon` (executed only if the customer possesses a valid coupon code).

Module IV: Verification, Validation & Testing — Complete 16-Topic Syllabus Tracker

Topic 40: Verification ("Are We Building the Product Right?")

Topic 42: V&V Planning & Test Strategy Life Cycle

Topic 44: Static Analysis (Control Flow, Data Flow & Linters)

Topic 46: Testing Functions & Test Harness Design

Topic 48: White-Box Testing (Statement & Branch Coverage)

Topic 50: Basis Path Testing & McCabe's Cyclomatic Complexity

Topic 52: Unit Testing (Stubs, Drivers & Test Doubles)

Topic 54: System Testing (Stress, Load, Recovery & Acceptance)

Topic 41: Validation ("Are We Building the Right Product?")

Topic 43: Software Inspection (Fagan Formal Defect Reviews)

Topic 45: Software Testing Foundations & Defect Taxonomies

Topic 47: Test Case Design (Formal Test Case Artifact Structure)

Topic 49: Black-Box Testing (Equivalence Partitioning & BVA)

Topic 51: Control Structure Testing (Condition & Loop Testing)

Topic 53: Integration Testing (Top-Down, Bottom-Up, Sandwich)

Topic 55: Software Reliability & Availability (MTBF, MTTF, MTTR)

Topic 40 – 44: Verification vs. Validation & Static Quality Techniques

Parameter	Verification (Static Quality Assurance)	Validation (Dynamic Quality Control)
Boehm's Question	"Are we building the product right?"	"Are we building the right product?"
Execution	Static evaluation <i>without executing code</i> (Inspections, Walkthroughs, Linters).	Dynamic execution of software using actual test cases.
Target Evaluation	Verifies whether design/code conforms to SRS specifications.	Validates whether the operational system satisfies actual user needs.

Topic 48 & 49: White-Box vs. Black-Box Testing Methodologies

Feature	Black-Box Testing (Functional)	White-Box Testing (Structural / Glass-Box)
Knowledge Needed	Zero internal code knowledge; tests external inputs vs. expected outputs.	Full knowledge of source code logic, branches, loops, and data structures.
Primary Techniques	• Equivalence Partitioning (EP): Divides inputs into valid/invalid equivalence classes. • Boundary Value Analysis (BVA): Tests boundaries ($\min$, $\min + 1$, nominal, $\max - 1$, $\max$).	• Statement Coverage, Branch Coverage, Condition Coverage. • Basis Path Testing via Control Flow Graphs (CFG).

Topic 50: Basis Path Testing & McCabe's Cyclomatic Complexity

M McCabe's Cyclomatic Complexity $V(G)$ Formulations (McCabe, 1976): 1. From Edges (E) and Nodes (N):

$V(G) = E - N + 2$ 2. From Predicate / Decision Nodes (P): $V(G) = P + 1$ 3. From Enclosed Regions (R): $V(G) = R$

Significance: $V(G)$ defines the exact number of linearly independent execution paths required to achieve 100% branch coverage!

Step-by-Step Solved Problem: Cyclomatic Complexity & Independent Paths

```
int find_max(int a, int b, int c) {  
    int max = a;           // Node 1  
    if (b > max)            // Node 2 (Predicate)  
        max = b;           // Node 3  
    if (c > max)            // Node 4 (Predicate)  
        max = c;           // Node 5  
    return max;            // Node 6  
}
```

Graph Analysis: Nodes $N = 6$, Edges $E = 7$, Predicates $P = 2$ (Nodes 2 and 4).

$$V(G) = E - N + 2 = 7 - 6 + 2 = \mathbf{3} \quad (\text{or } P + 1 = 2 + 1 = \mathbf{3})$$

Set of 3 Linearly Independent Paths:

- **Path 1:** $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$ (Test: $a = 10, b = 5, c = 2$)
- **Path 2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6$ (Test: $a = 5, b = 10, c = 2$)
- **Path 3:** $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6$ (Test: $a = 5, b = 2, c = 10$)

Topic 52 – 55: Levels of Testing & Software Reliability

Software Reliability & Availability Formulas:

Mean Time Between Failures (MTBF) = Mean Time To Failure (MTTF) + Mean Time To Repair (MTTR)

$$\text{Availability (A)} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

M4 Active Recall & Exam Questions

Q1. Explain Top-Down vs. Bottom-Up Integration Testing. Define Stubs and Drivers with diagrams. (10 Marks)

- **Top-Down Integration:** Modules are integrated from top to bottom of the control hierarchy. Subordinate modules not yet developed are simulated using **Stubs** (dummy subroutines called by the tested module).
- **Bottom-Up Integration:** Low-level worker modules are tested first. Superordinate modules not yet developed are simulated using **Drivers** (test harnesses that invoke the module and display results).
- **Sandwich (Hybrid) Integration:** Combines top-down for top layers and bottom-up for low-level utility libraries, minimizing stub/driver development overhead.

Module V: Project Management, Quality & Maintenance — Complete 13-Topic Tracker

Topic 56: Process & Product Metrics (Defect Density & Productivity)

Topic 58: Software Project Estimation Paradigms & Heuristics

Topic 60: Empirical Estimation Models & Historical Calibration

Topic 62: Function Point Analysis (5 Function Types: EI, EO, EQ, ILF, EIF)

Topic 64: Quality Planning & SQA Checklists

Topic 66: Software Configuration Management (SCM, Baselines & CCB)

Topic 68: Software Re-Engineering (Reverse & Forward Engineering)

Topic 57: Software Measurement (Lines of Code & Token Metrics)

Topic 59: Decomposition Techniques (Problem vs. Process Decomposition)

Topic 61: The COCOMO Model (Organic, Semi-Detached, Embedded)

Topic 63: Quality Assurance & Standards (ISO 9001 & CMMI 5 Levels)

Topic 65: Quality Control (QA vs. QC: Prevention vs. Detection)

Topic 67: Software Maintenance (The 4 CAP-P Maintenance Types)

Topic 61: Boehm's COCOMO Model (Constructive Cost Model)

Basic COCOMO Equations (Barry Boehm, 1981):

$$\text{Effort } (E) = a \cdot (\text{KLOC})^b \quad [\text{Person-Months (PM)}]$$

$$\text{Development Time } (T_{\text{dev}}) = c \cdot (E)^d \quad [\text{Months}]$$

$$\text{Average Staffing Size } (SS) = \frac{E}{T_{\text{dev}}} \quad [\text{Persons}], \quad \text{Productivity} = \frac{\text{KLOC}}{E} \quad [\text{KLOC/PM}]$$

Project Mode	Team & Environment Characteristics	Effort (a, b)	Time (c, d)
1. Organic	Small, experienced teams working with flexible requirements in familiar in-house environment.	$E = 2.4(\text{KLOC})^{1.05}$	$T_{\text{dev}} = 2.5(E)^{0.38}$
2. Semi-Detached	Medium teams with mixed experience levels working under intermediate constraints.	$E = 3.0(\text{KLOC})^{1.12}$	$T_{\text{dev}} = 2.5(E)^{0.35}$
3. Embedded	Extremely tight, rigid hardware/software interface constraints (e.g., flight control, medical systems).	$E = 3.6(\text{KLOC})^{1.20}$	$T_{\text{dev}} = 2.5(E)^{0.32}$

Topic 62: Function Point (FP) Analysis (Albrecht, 1979)

Function Point Mathematical Formulations:

$$\text{FP} = \text{UFP} \times \text{VAF} = \text{UFP} \times \left[0.65 + 0.01 \times \sum_{i=1}^{14} C_i \right]$$

Where **UFP** is Unadjusted Function Points, and $\sum C_i$ is the sum of 14 General System Characteristics (TDI: 0 to 70).

Function Type	Low Weight	Average Weight	High Weight	Description
1. External Inputs (EI)	3	4	6	User transactions adding/updating internal DB records.
2. External Outputs (EO)	4	5	7	Calculated reports/displays leaving system boundary.
3. External Inquiries (EQ)	3	4	6	Interactive queries retrieving direct data without calculation.
4. Internal Logical Files (ILF)	7	10	15	Major database tables maintained inside system boundary.
5. External Interface Files (EIF)	5	7	10	Data files maintained by another external system.

Topic 63: Capability Maturity Model Integration (CMMI Levels 1 – 5)

CMMI Maturity Level	Process Predictability & Characteristics
Level 1: Initial	Ad-hoc, chaotic, undocumented; success depends on individual heroic developer efforts.
Level 2: Managed	Basic project management established; requirements, costs, and schedules are tracked and repeatable.
Level 3: Defined	Standard engineering processes documented and standardized across entire organization.
Level 4: Quantitatively Managed	Process and product quality quantitatively measured, controlled, and statistically tracked.
Level 5: Optimizing	Continuous process improvement driven by quantitative feedback and innovative technology piloting.

Topic 66 – 68: SCM & The 4 Types of Software Maintenance (CAP-P)

Memory Hook: The 4 Maintenance Types (CAP-P)

- 1. **Corrective Maintenance (20%)**: Reactive fixing of residual bugs discovered in production.
- 2. **Adaptive Maintenance (25%)**: Modifying software to operate in new environments (OS upgrade, cloud migration).
- 3. **Perfective Maintenance (50% — Largest)**: Enhancing existing functionality, performance, and user convenience.
- 4. **Preventive Maintenance (5%)**: Refactoring code to enhance maintainability and prevent future software decay.

M5 Active Recall & Exam Questions

Q1. A 30 KLOC software is developed in Organic mode. Calculate Effort, Development Time, Staff Size, and Productivity using Basic COCOMO. (8 Marks)

Given: Size = 30 KLOC, Mode: Organic ($a = 2.4, b = 1.05, c = 2.5, d = 0.38$).

- Effort $E = 2.4 \times (30)^{1.05} = 2.4 \times 35.64 \approx \mathbf{85.54}$ Person-Months.
- Development Time $T_{dev} = 2.5 \times (85.54)^{0.38} = 2.5 \times 5.43 \approx \mathbf{13.58}$ Months.
- Average Staff Size $SS = \frac{E}{T_{dev}} = \frac{85.54}{13.58} \approx \mathbf{6.3}$ Persons.
- Productivity $= \frac{30}{85.54} \approx \mathbf{0.351}$ KLOC/PM (**351 LOC/PM**).